

```

import seaborn as sns
import pandas as pd

penguins = sns.load_dataset("penguins")
print(penguins.columns)
print("First 5 rows:")
print(penguins.head())

numeric_cols = penguins.select_dtypes(include="number").columns

summary = pd.DataFrame(index=numeric_cols)
summary["Count"] = penguins[numeric_cols].count()
summary["Mean"] = penguins[numeric_cols].mean()
summary["Median"] = penguins[numeric_cols].median()
summary["Std Dev"] = penguins[numeric_cols].std()

summary["IQR"] = (
    penguins[numeric_cols].quantile(0.75)
    - penguins[numeric_cols].quantile(0.25)
)
summary["Skewness"] = penguins[numeric_cols].skew()
summary["Kurtosis"] = penguins[numeric_cols].kurtosis()
print("\nDescriptive Statistics:")
print(summary.round(3))

species_means = penguins.groupby("species")[numeric_cols].mean()
print("\nSpecies-wise Mean Comparison:")
print(species_means.round(2))

```

```

Index(['species', 'island', 'bill_length_mm', 'bill_depth_mm',
      'flipper_length_mm', 'body_mass_g', 'sex'],
      dtype='object')

```

First 5 rows:

	species	island	bill_length_mm	bill_depth_mm	flipper_length_mm
0	Adelie	Torgersen	39.1	18.7	181
1	Adelie	Torgersen	39.5	17.4	186
2	Adelie	Torgersen	40.3	18.0	195
3	Adelie	Torgersen	NaN	NaN	NaN
4	Adelie	Torgersen	36.7	19.3	193

	body_mass_g	sex
0	3750.0	Male
1	3800.0	Female
2	3250.0	Female
3	NaN	NaN
4	3450.0	Female

Descriptive Statistics:

Count	Mean	Median	Std Dev	IQR	Skew
-------	------	--------	---------	-----	------

bill_length_mm	342	43.922	44.45	5.460	9.275	0
bill_depth_mm	342	17.151	17.30	1.975	3.100	-0
flipper_length_mm	342	200.915	197.00	14.062	23.000	0
body_mass_g	342	4201.754	4050.00	801.955	1200.000	0

	Kurtosis
bill_length_mm	-0.876
bill_depth_mm	-0.907
flipper_length_mm	-0.984
body_mass_g	-0.719

Species-wise Mean Comparison:

	bill_length_mm	bill_depth_mm	flipper_length_mm	body_ma
species				
Adelie	38.79	18.35	189.95	370
Chinstrap	48.83	18.42	195.82	373
Gentoo	47.50	14.98	217.19	507

```
import seaborn as sns
import matplotlib.pyplot as plt

penguins = sns.load_dataset("penguins")

plt.figure(figsize=(8, 5))
sns.violinplot(
    data=penguins,
    x="species",
    y="flipper_length_mm",
    inner="box"
)
plt.title("Flipper Length by Penguin Species")
plt.xlabel("Species")
plt.ylabel("Flipper Length (mm)")
plt.show()
```

```
import seaborn as sns
import matplotlib.pyplot as plt

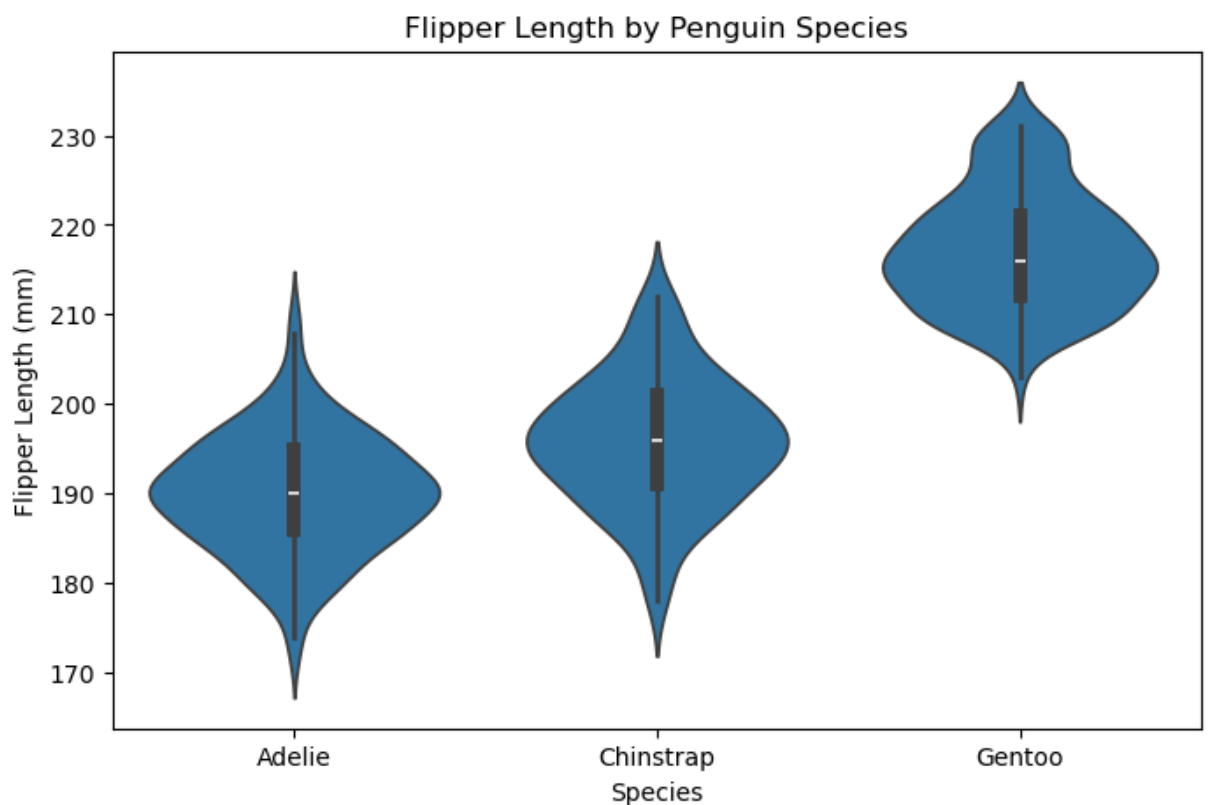
penguins = sns.load_dataset("penguins")

plt.figure(figsize=(8, 5))
sns.violinplot(
    data=penguins,
    x="species",
    y="flipper_length_mm",
    inner="box"
)
plt.title("Flipper Length by Penguin Species")
plt.xlabel("Species")
plt.ylabel("Flipper Length (mm)")
plt.tight_layout()
```

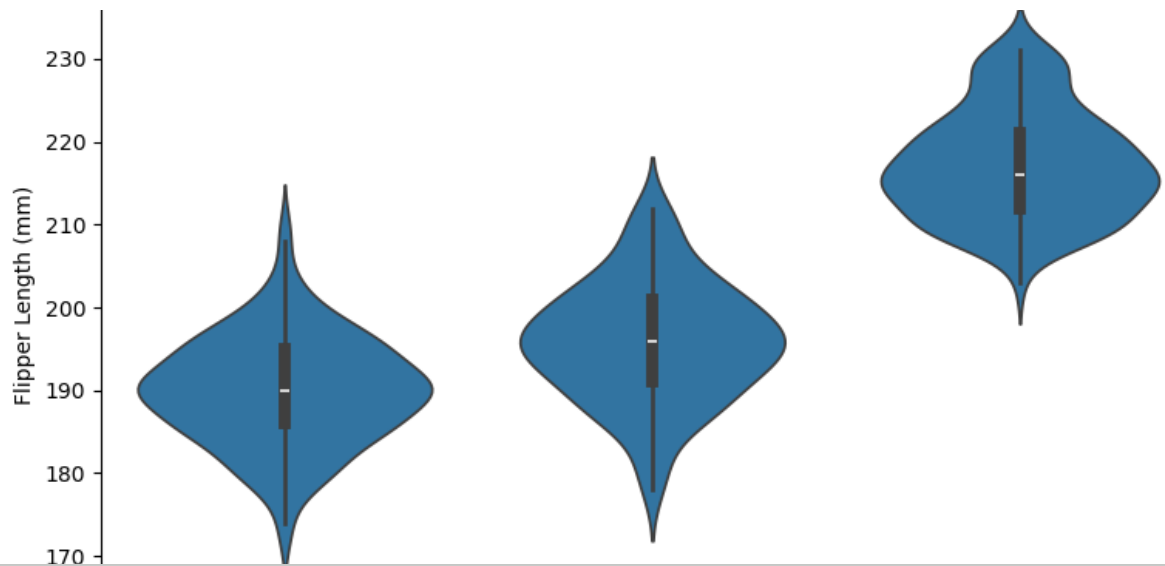
```
plt.show()

mean_mass = (
    penguins
    .groupby(["species", "sex"], observed=True)["body_mass_g"]
    .mean()
    .reset_index()
)
plt.figure(figsize=(9, 5))
sns.barplot(
    data=mean_mass,
    x="species",
    y="body_mass_g",
    hue="sex"
)
plt.title("Mean Body Mass by Species and Sex")
plt.xlabel("Species")
plt.ylabel("Mean Body Mass (g)")
plt.tight_layout()
plt.show()

print("Mean Body Mass by Species and Sex:")
print(
    mean_mass
    .pivot(index="species", columns="sex", values="body_mass_g")
    .round(2)
)
```



Flipper Length by Penguin Species



```
import seaborn as sns
import matplotlib.pyplot as plt
from scipy import stats

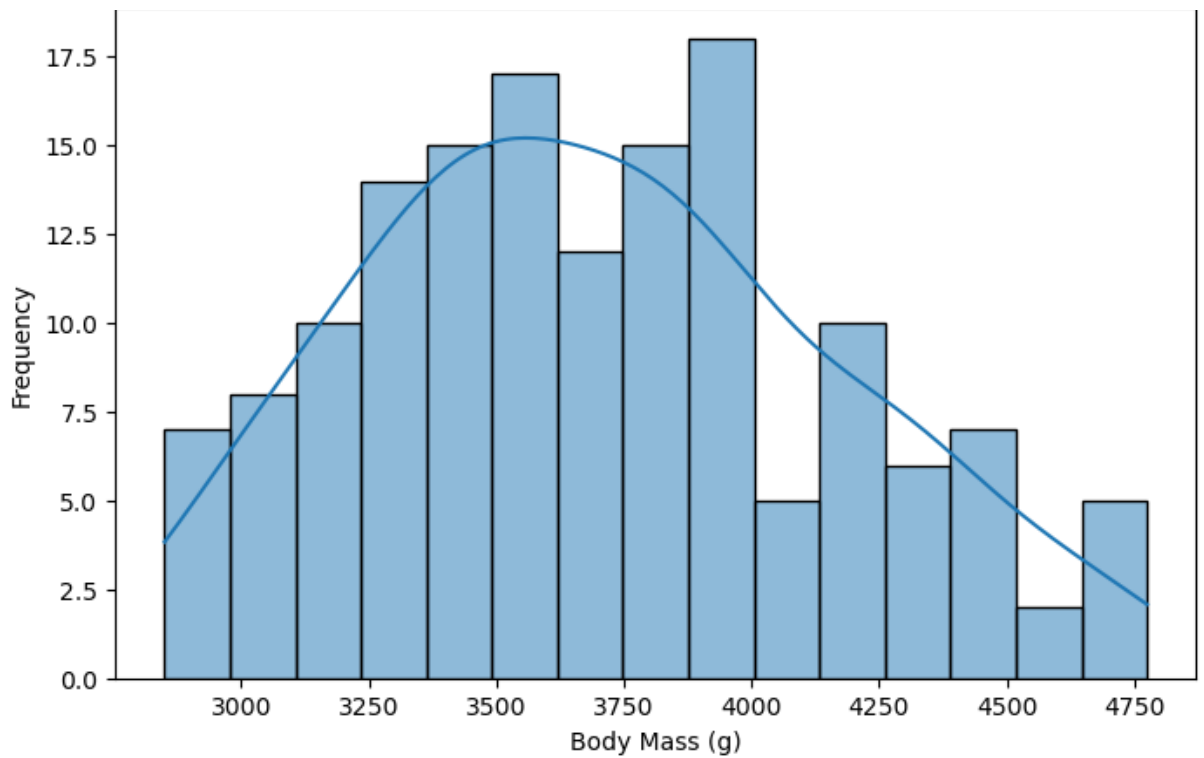
penguins = sns.load_dataset("penguins")
adelie = penguins[penguins["species"] == "Adelie"]["body_mass_g"].d

plt.figure(figsize=(8, 5))
sns.histplot(adelie, bins=15, kde=True)
plt.title("Histogram + KDE of Adelie Body Mass")
plt.xlabel("Body Mass (g)")
plt.ylabel("Frequency")
plt.show()

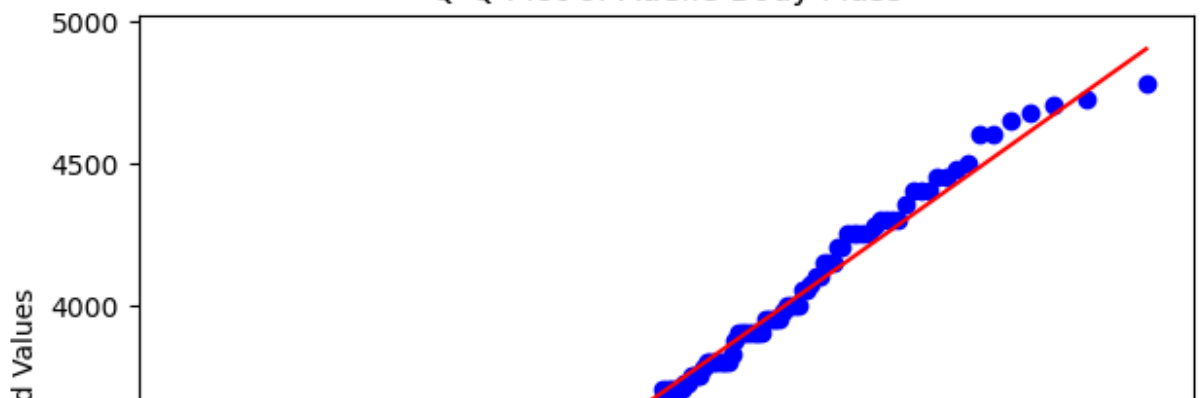
plt.figure(figsize=(7, 5))
stats.probplot(adelie, dist="norm", plot=plt)
plt.title("Q-Q Plot of Adelie Body Mass")
plt.xlabel("Theoretical Quantiles")
plt.ylabel("Ordered Values")
plt.show()

stat, p_value = stats.shapiro(adelie)
print("Shapiro-Wilk Test")
print("-----")
print("W statistic:", stat)
print("p-value:", p_value)

if p_value > 0.05:
    print("\nConclusion: Fail to reject H0.")
    print("Adelie body mass is approximately normally distributed.")
else:
    print("\nConclusion: Reject H0.")
    print("Adelie body mass is not normally distributed.")
```



Q-Q Plot of Adelie Body Mass



```
import seaborn as sns
from scipy import stats
import numpy as np

penguins = sns.load_dataset("penguins")
adelie = penguins[penguins["species"] == "Adelie"]["body_mass_g"].d
chinstrap = penguins[penguins["species"] == "Chinstrap"]["body_mass_g"].d
gentoo = penguins[penguins["species"] == "Gentoo"]["body_mass_g"].d
alpha = 0.05

t_stat_a, p_value_a = stats.ttest_1samp(adelie, 3600)
print("===== (a) One-Sample t-Test =====")
print("Adelie mean body mass:", adelie.mean(), "g")
print("t-statistic:", t_stat_a)
print("p-value:", p_value_a)
if p_value_a < alpha:
    print("Decision: Reject H0")
    print("Conclusion: The mean Adelie body mass is significantly d
else:
```

```

    print("Decision: Fail to reject H0")
    print("Conclusion: There is no significant evidence that the me

t_stat_b, p_value_b = stats.ttest_ind(
    adelie,
    chinstrap,
    equal_var=False
)

mean_adelie = adelie.mean()
mean_chinstrap = chinstrap.mean()
sd_adelie = adelie.std(ddof=1)
sd_chinstrap = chinstrap.std(ddof=1)
n_adelie = len(adelie)
n_chinstrap = len(chinstrap)

pooled_sd = np.sqrt(
    ((n_adelie - 1) * sd_adelie**2 +
     (n_chinstrap - 1) * sd_chinstrap**2)
    / (n_adelie + n_chinstrap - 2)
)
cohens_d = (mean_adelie - mean_chinstrap) / pooled_sd
print("\n===== (b) Welch t-Test =====")
print("Adelie mean:", mean_adelie, "g")
print("Chinstrap mean:", mean_chinstrap, "g")
print("t-statistic:", t_stat_b)
print("p-value:", p_value_b)
print("Cohen's d:", cohens_d)
if p_value_b < alpha:
    print("Decision: Reject H0")
    print("Conclusion: Adelie and Chinstrap mean body masses are si
else:
    print("Decision: Fail to reject H0")
    print("Conclusion: No significant difference between Adelie and

t_stat_c, p_value_c = stats.ttest_ind(
    adelie,
    gentoo,
    equal_var=False
)
print("\n===== (c) Welch t-Test =====")
print("Adelie mean:", mean_adelie, "g")
print("Gentoo mean:", gentoo.mean(), "g")
print("t-statistic:", t_stat_c)
print("p-value:", p_value_c)
if p_value_c < alpha:
    print("Decision: Reject H0")
    print("Conclusion: Adelie and Gentoo mean body masses are signi

```

```

else:
    print("Decision: Fail to reject H0")
    print("Conclusion: No significant difference between Adelie and

u_stat, p_value_u = stats.mannwhitneyu(
    adelie,
    chinstrap,
    alternative="two-sided"
)
print("\n===== Mann-Whitney U Test =====")
print("U-statistic:", u_stat)
print("p-value:", p_value_u)
if p_value_u < alpha:
    print("Decision: Reject H0")
    print("Conclusion: Adelie and Chinstrap body masses differ sign
else:
    print("Decision: Fail to reject H0")
    print("Conclusion: No significant difference between Adelie and

```

```

===== (a) One-Sample t-Test =====
Adelie mean body mass: 3700.662251655629 g
t-statistic: 2.6974483884458738
p-value: 0.00778748518472283
Decision: Reject H0
Conclusion: The mean Adelie body mass is significantly different fro

```

```

===== (b) Welch t-Test =====
Adelie mean: 3700.662251655629 g
Chinstrap mean: 3733.0882352941176 g
t-statistic: -0.5430902472363278
p-value: 0.5878608159584937
Cohen's d: -0.07420226465605512
Decision: Fail to reject H0
Conclusion: No significant difference between Adelie and Chinstrap m

```

```

===== (c) Welch t-Test =====
Adelie mean: 3700.662251655629 g
Gentoo mean: 5076.016260162602 g
t-statistic: -23.38602766996551
p-value: 7.709823145147335e-65
Decision: Reject H0
Conclusion: Adelie and Gentoo mean body masses are significantly dif

```

```

===== Mann-Whitney U Test =====
U-statistic: 4831.0
p-value: 0.4854772721011861
Decision: Fail to reject H0
Conclusion: No significant difference between Adelie and Chinstrap b

```

```

import seaborn as sns
from scipy import stats

```

```

from statsmodels.stats.multicomp import pairwise_tukeyhsd

penguins = sns.load_dataset("penguins")
data = penguins[["species", "flipper_length_mm"]].dropna()

print("Mean Flipper Length by Species")
print("-----")
print(data.groupby("species")["flipper_length_mm"].mean())

adelie = data[data["species"] == "Adelie"]["flipper_length_mm"]
chinstrap = data[data["species"] == "Chinstrap"]["flipper_length_mm"]
gentoo = data[data["species"] == "Gentoo"]["flipper_length_mm"]

f_stat, p_value = stats.f_oneway(
    adelie,
    chinstrap,
    gentoo
)
print("\nOne-Way ANOVA")
print("-----")
print("F-statistic:", f_stat)
print("p-value:", p_value)
alpha = 0.05
if p_value < alpha:
    print("Decision: Reject H0")
    print("Conclusion: Mean flipper length differs among the specie
else:
    print("Decision: Fail to reject H0")
    print("Conclusion: No significant difference among species.")

tukey = pairwise_tukeyhsd(
    endog=data["flipper_length_mm"],
    groups=data["species"],
    alpha=0.05
)
print("\nTukey's HSD Test")
print("-----")
print(tukey)

print("\nSignificant Pairwise Differences:")
for row in tukey.summary().data[1:]:
    group1, group2, meandiff, p_adj, lower, upper, reject = row

    if reject:
        print(f"{group1} vs {group2}: Significant difference")
    else:
        print(f"{group1} vs {group2}: No significant difference")

```

Mean Flipper Length by Species

```
species
Adelie      189.953642
Chinstrap   195.823529
Gentoo      217.186992
Name: flipper_length_mm, dtype: float64
```

One-Way ANOVA

```
F-statistic: 594.8016274385172
p-value: 1.3517103385730013e-111
Decision: Reject H0
Conclusion: Mean flipper length differs among the species.
```

Tukey's HSD Test

```
Multiple Comparison of Means - Tukey HSD, FWER=0.05
=====
group1 group2 meandiff p-adj lower upper reject
-----
Adelie Chinstrap 5.8699 0.0 3.5866 8.1532 True
Adelie Gentoo 27.2333 0.0 25.3344 29.1323 True
Chinstrap Gentoo 21.3635 0.0 19.0008 23.7261 True
=====
```

Significant Pairwise Differences:

```
Adelie vs Chinstrap: Significant difference
Adelie vs Gentoo: Significant difference
Chinstrap vs Gentoo: Significant difference
```

```
import seaborn as sns
import numpy as np
from scipy import stats

penguins = sns.load_dataset("penguins")
titanic = sns.load_dataset("titanic")

gentoo = penguins[penguins["species"] == "Gentoo"]["body_mass_g"].d
n = len(gentoo)
mean_gentoo = gentoo.mean()
std_gentoo = gentoo.std(ddof=1)
se = std_gentoo / np.sqrt(n)
t_critical = stats.t.ppf(0.975, df=n-1)
margin_error = t_critical * se
ci_mean = (
    mean_gentoo - margin_error,
    mean_gentoo + margin_error
)
print("A. 95% CI for Mean Gentoo Body Mass")
```

```

print("-----")
print("Sample size:", n)
print("Mean:", mean_gentoo)
print("95% CI:", ci_mean)

survived = titanic["survived"].dropna()
n_titanic = len(survived)
survival_count = survived.sum()
p_hat = survival_count / n_titanic

se_prop = np.sqrt((p_hat * (1 - p_hat)) / n_titanic)

z_critical = stats.norm.ppf(0.975)
margin_prop = z_critical * se_prop
ci_prop = (
    p_hat - margin_prop,
    p_hat + margin_prop
)
print("\nB. 95% CI for Titanic Survival Proportion")
print("-----")
print("Total passengers:", n_titanic)
print("Survived:", survival_count)
print("Survival proportion:", p_hat)
print("Survival percentage:", p_hat * 100)
print("95% CI:", ci_prop)
print("95% CI (%):", ci_prop[0] * 100, "% to", ci_prop[1] * 100, "%

np.random.seed(42)
B = 10000 # 10,000 bootstrap resamples
bootstrap_medians = []
for i in range(B):
    sample = np.random.choice(
        gentoo,
        size=len(gentoo),
        replace=True
    )
    bootstrap_medians.append(np.median(sample))
bootstrap_medians = np.array(bootstrap_medians)

ci_median = np.percentile(
    bootstrap_medians,
    [2.5, 97.5]
)
median_gentoo = np.median(gentoo)
print("\nC. 95% Bootstrap CI for Median Gentoo Body Mass")
print("-----")
print("Median:", median_gentoo)

```

```

print("Number of bootstrap resamples:", B)
print("95% Bootstrap CI:", ci_median)

print("\nINTERPRETATIONS")
print("-----")
print(
    f"(a) We are 95% confident that the population mean "
    f"Gentoo body mass lies between {ci_mean[0]:.2f} g and "
    f"{ci_mean[1]:.2f} g."
)
print(
    f"(b) We are 95% confident that the population survival "
    f"proportion of Titanic passengers lies between "
    f"{ci_prop[0]*100:.2f}% and {ci_prop[1]*100:.2f}%."
)
print(
    f"(c) We are 95% confident that the population median "
    f"Gentoo body mass lies between {ci_median[0]:.2f} g and "
    f"{ci_median[1]:.2f} g, based on 10,000 bootstrap resamples."
)

```

A. 95% CI for Mean Gentoo Body Mass

```

-----
Sample size: 123
Mean: 5076.016260162602
95% CI: (4986.0342795578745, 5165.998240767329)

```

B. 95% CI for Titanic Survival Proportion

```

-----
Total passengers: 891
Survived: 342
Survival proportion: 0.3838383838383838
Survival percentage: 38.38383838383838
95% CI: (0.3519060427032577, 0.4157707249735099)
95% CI (%): 35.19060427032577 % to 41.57707249735099 %

```

C. 95% Bootstrap CI for Median Gentoo Body Mass

```

-----
Median: 5000.0
Number of bootstrap resamples: 10000
95% Bootstrap CI: [4900. 5200.]

```

INTERPRETATIONS

```

-----
(a) We are 95% confident that the population mean Gentoo body mass l
(b) We are 95% confident that the population survival proportion of
(c) We are 95% confident that the population median Gentoo body mass

```

```

import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

```

```

import statsmodels.api as sm

penguins = sns.load_dataset("penguins")

numeric_data = penguins.select_dtypes(include="number")

correlation = numeric_data.corr(method="pearson")
print("Pearson Correlation Matrix")
print("-----")
print(correlation)

plt.figure(figsize=(8, 6))
sns.heatmap(
    correlation,
    annot=True,
    cmap="coolwarm",
    fmt=".2f",
    linewidths=0.5
)
plt.title("Pearson Correlation Heatmap of Penguin Numeric Variables")
plt.show()

data = penguins[
    ["flipper_length_mm", "body_mass_g"]
].dropna()
plt.figure(figsize=(8, 6))
sns.regplot(
    data=data,
    x="flipper_length_mm",
    y="body_mass_g"
)
plt.title("Body Mass vs Flipper Length")
plt.xlabel("Flipper Length (mm)")
plt.ylabel("Body Mass (g)")
plt.show()

X = data["flipper_length_mm"]

y = data["body_mass_g"]

X = sm.add_constant(X)

model = sm.OLS(y, X).fit()

print("\nOLS Regression Results")
print("-----")
print(model.summary())

```

```

intercept = model.params["const"]
slope = model.params["flipper_length_mm"]
r_squared = model.rsquared
print("\nKey Results")
print("-----")
print("Intercept:", intercept)
print("Slope:", slope)
print("R-squared:", r_squared)
print("\nInterpretation:")
print(
    f"For every 1 mm increase in flipper length, "
    f"predicted body mass increases by approximately "
    f"{slope:.2f} grams."
)
print(
    f"R-squared = {r_squared:.3f}, meaning approximately "
    f"{r_squared * 100:.1f}% of the variation in body mass "
    f"is explained by flipper length in this linear model."
)

```

Pearson Correlation Matrix

```

-----
                bill_length_mm  bill_depth_mm  flipper_length_mm
bill_length_mm          1.000000         -0.235053          0.656181
bill_depth_mm          -0.235053          1.000000         -0.583851
flipper_length_mm       0.656181         -0.583851          1.000000
body_mass_g            0.595110         -0.471916          0.871202

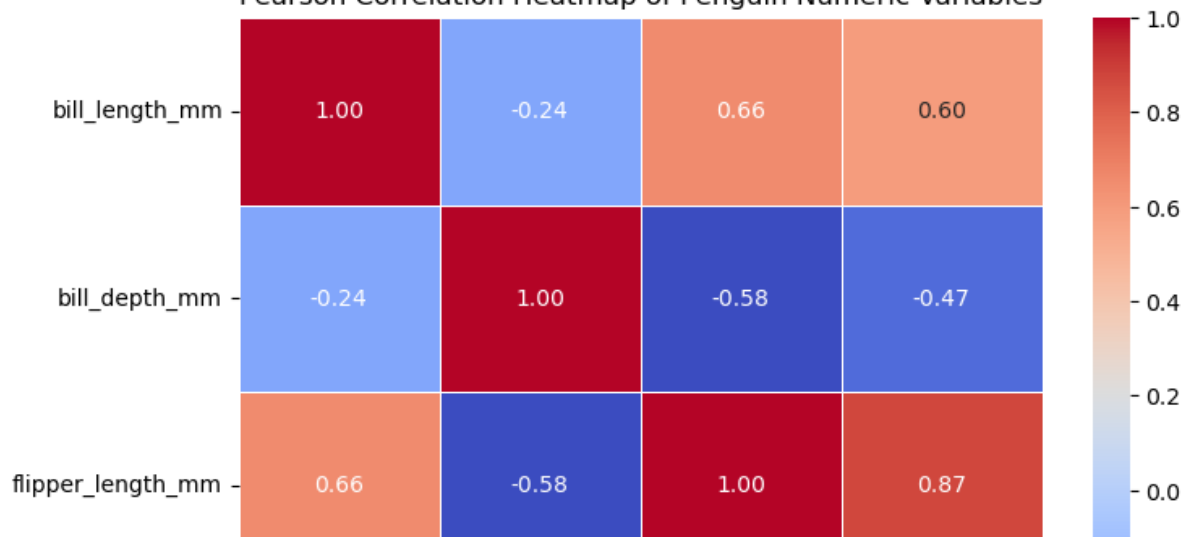
```

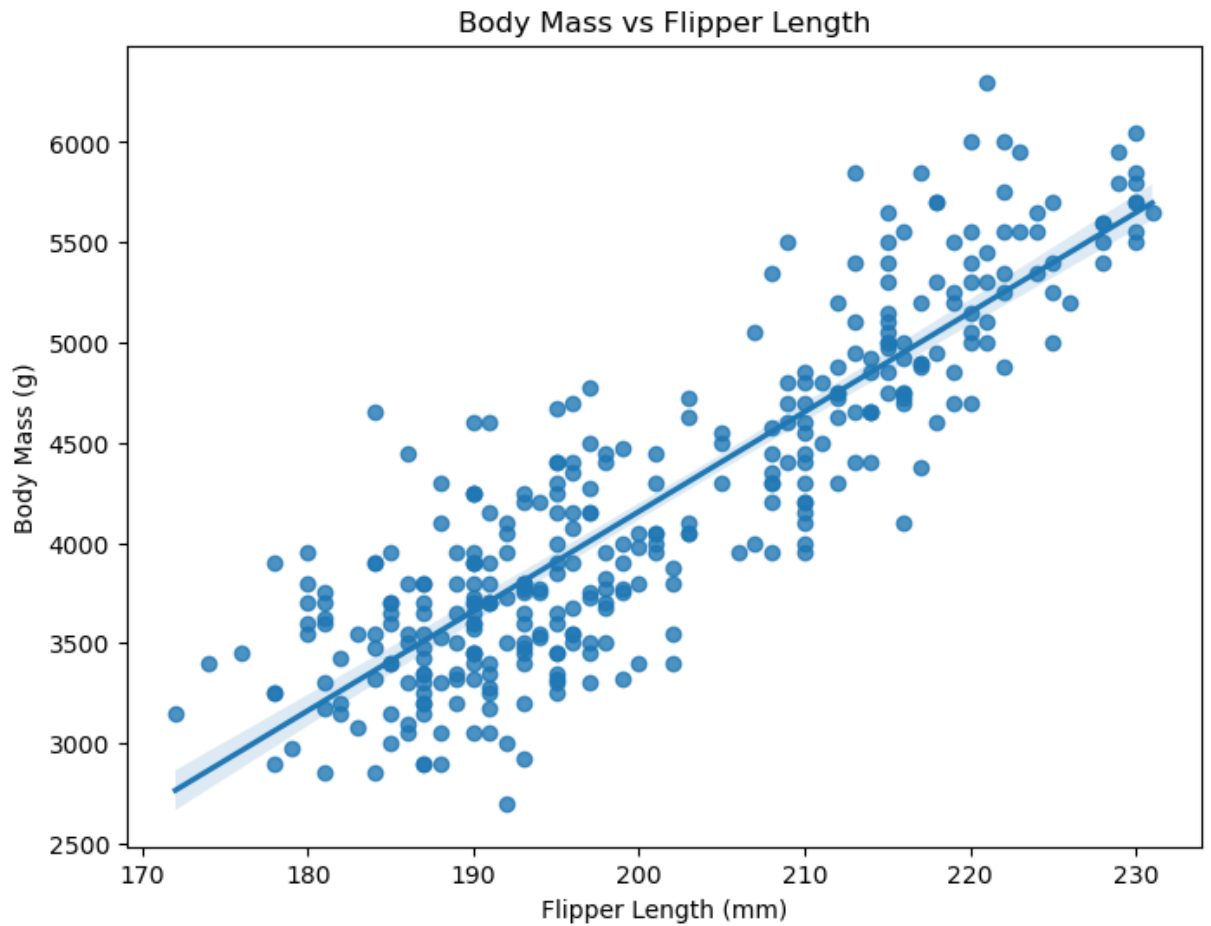
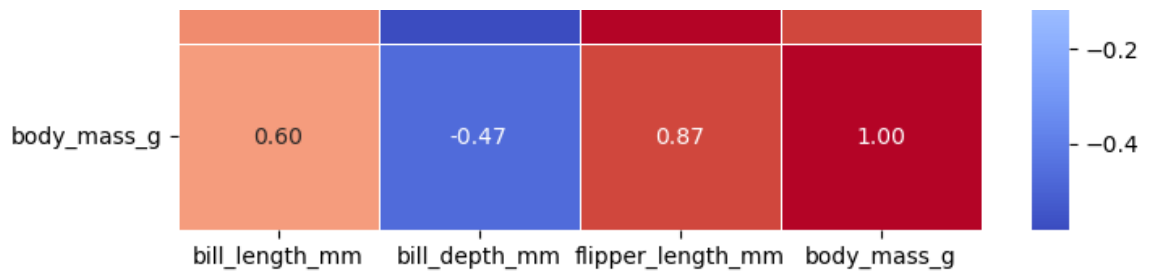
```

                body_mass_g
bill_length_mm      0.595110
bill_depth_mm     -0.471916
flipper_length_mm   0.871202
body_mass_g        1.000000

```

Pearson Correlation Heatmap of Penguin Numeric Variables





OLS Regression Results

OLS Regression Results					
=====					
Dep. Variable:	body_mass_g	R-squared:			
Model:	OLS	Adj. R-squared:			
Method:	Least Squares	F-statistic:			
Date:	Mon, 05 Oct 2026	Prob (F-statistic):			
Time:	10:50:28	Log-Likelihood:			
No. Observations:	342	AIC:			
Df Residuals:	340	BIC:			
Df Model:	1				
Covariance Type:	nonrobust				
=====					
	coef	std err	t	P> t	[

const	-5780.8314	305.815	-18.903	0.000	-638
flipper_length_mm	49.6856	1.518	32.722	0.000	4
=====					

Start coding or generate with AI.